



Convoluzione Parallela di Immagini con Filtro Gaussiano

Analisi e progressione di un codice HPC: dal Sequenziale all'Ibrido

Kamil D'Anna, 1000022172

Il Filtro Gaussiano

La convoluzione è un'operazione matematica che ricalcola il valore di ogni singolo pixel basandosi su una matrice di pesi chiamata *Kernel*.

Per simulare una sfocatura naturale (blur), il kernel segue una distribuzione Gaussiana, assegnando maggior peso al pixel

$$val = \frac{\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)}{2\pi\sigma^2}$$

La matrice viene infine normalizzata dividendo ogni elemento per la somma totale, garantendo che la luminosità complessiva dell'immagine rimanga inalterata.

Implementazione Sequenziale

L'algoritmo base si fonda su 4 cicli annidati: due esterni per percorrere le dimensioni dell'immagine ($Y \times X$) e due interni per applicare la maschera del kernel.

Complessità: Il numero di moltiplicazioni per singolo pixel cresce in proporzione all'area della maschera, con complessità asintotica $O(K^2)$.

Gestione dei bordi: Per evitare errori di segmentazione ai confini dell'immagine, la funzione di `clamp` replica i valori dei pixel marginali.

Specifiche dell'ambiente

I test sono stati eseguiti su un singolo nodo di calcolo con le seguenti caratteristiche:

- **CPU:** AMD Ryzen 7 5000 Series
- **Core:** 8 core fisici, 16 thread logici
- **Compilatore:** GCC con flag di massima ottimizzazione -O3

Performance della Baseline (Kernel 5x5): Il tempo di esecuzione sequenziale per il filtraggio completo dell'immagine di input 4096x4096 si attesta a **0,40 secondi**, mentre 8192x8192 si attesta a **1,46 secondi**.

Schema di parallelizzazione

Livello MPI: Distribuzione a "grana grossa".

L'immagine viene suddivisa in blocchi orizzontali assegnati a processi indipendenti.

Livello OpenMP: Distribuzione a "grana fine" su memoria condivisa.
All'interno di ogni blocco, i thread si spartiscono il calcolo delle righe in modo concorrente.

MPI e Halo Exchange

La divisione dell'immagine in blocchi isolati genera un problema ai confini:

i pixel marginali necessitano dei dati appartenenti al processo adiacente.

Soluzione: Ogni processo alloca un buffer maggiorato (padded).

Comunicazione: Tramite `MPI_Sendrecv`, i processi scambiano in modo sincrono le proprie righe di confine ("halo") con i vicini logici prima di iniziare il calcolo.

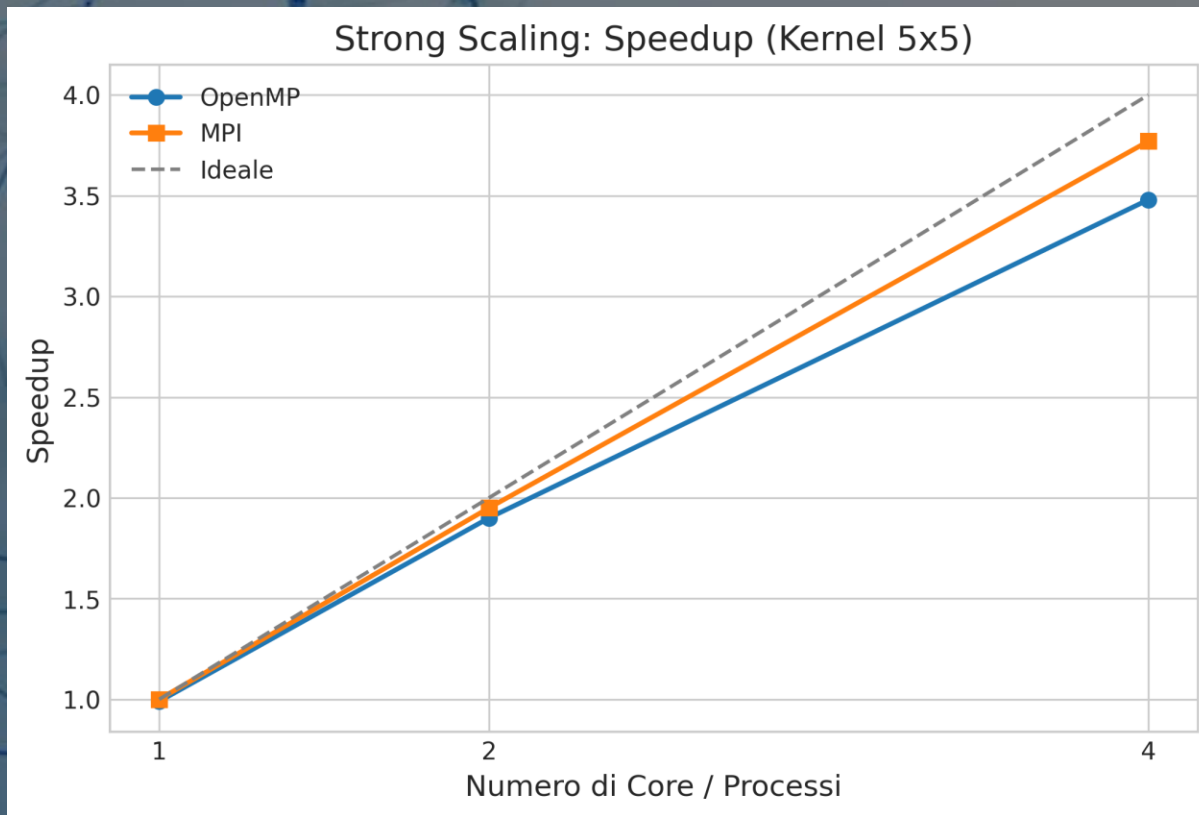
Architettura Ibrida

- L'unione dei due paradigmi richiede un'inizializzazione specifica dell'ambiente di memoria distribuita:

L'uso di `MPI_Init_thread` con livello `MPI_THREAD_FUNNELED` garantisce che solo il thread master effettui le chiamate di rete.

Il loop computazionale locale viene poi accelerato tramite le direttive standard `#pragma omp parallel for`.

Strong Scaling (Kernel 5x5)



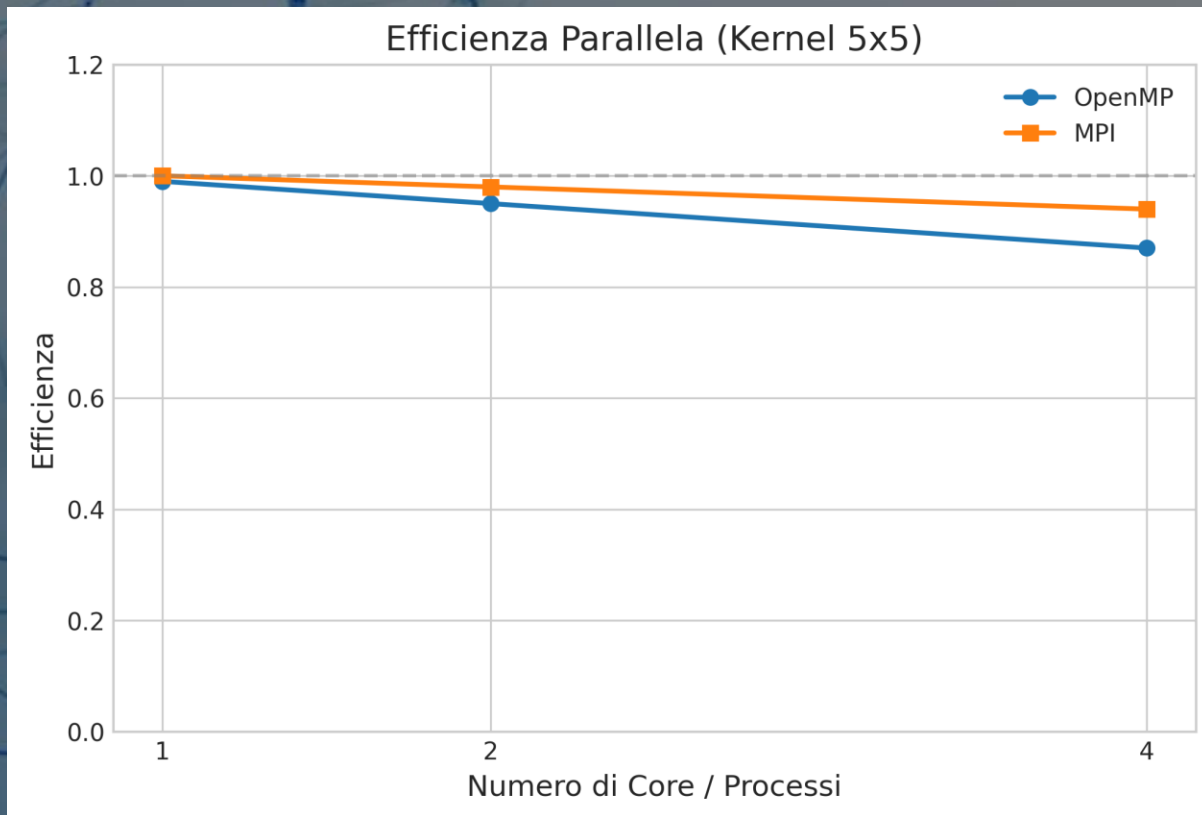
Fissata la dimensione del problema, MPI mostra una scalabilità superiore rispetto a OpenMP.

A 4 unità di calcolo, MPI raggiunge uno Speedup di **3.77x**, sfiorando il limite ideale (4.0x).

OpenMP si ferma a **3.47x**.

La memoria distribuita di MPI annulla i problemi di competizione sui bus di memoria (falso sharing) tipici degli ambienti fortemente condivisi.

Efficienza Parallela

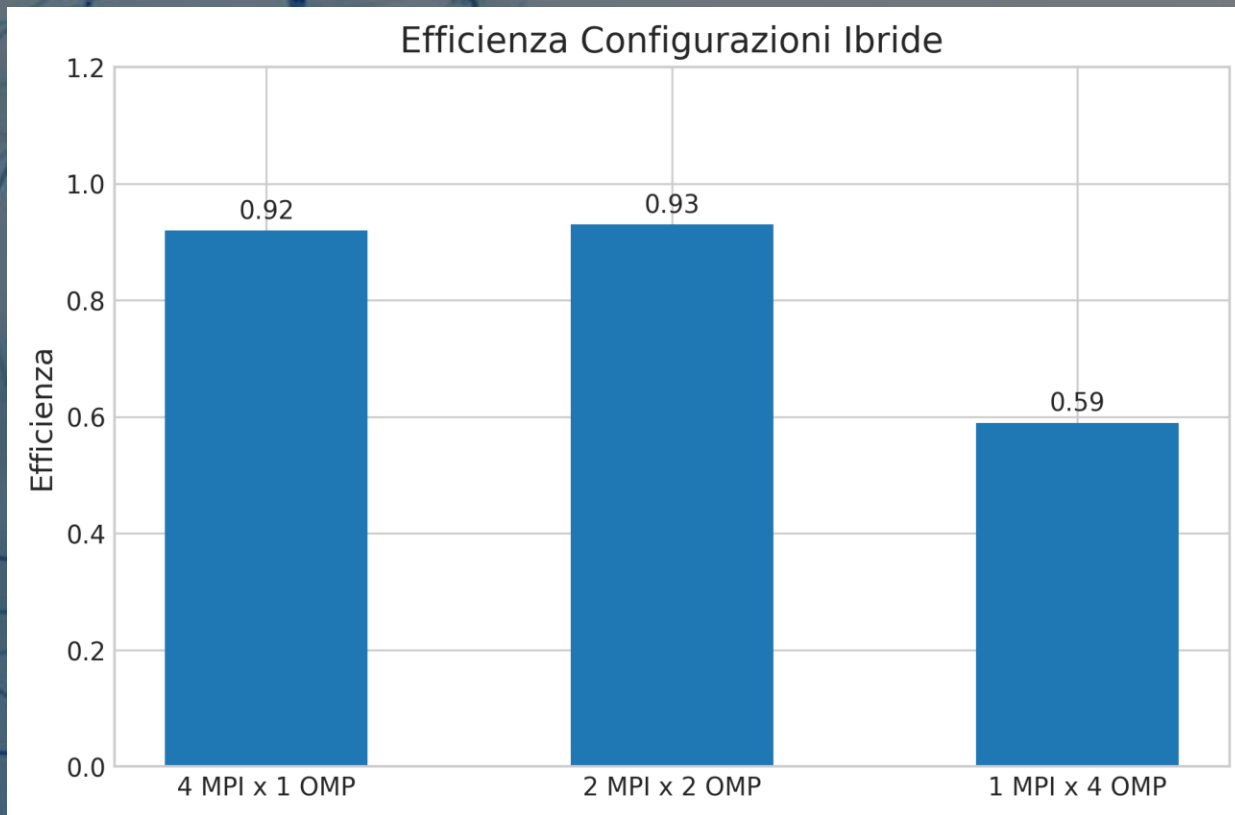


L'efficienza di MPI a 4 processi si mantiene eccellente, attestandosi al **94.3%**.

L'efficienza di OpenMP cala fisiologicamente all'**86.9%** a 4 thread, a causa dell'overhead di gestione della memoria concorrente.

Entrambi i modelli dimostrano un ottimo sfruttamento dell'hardware, con perdite minime rispetto ai limiti imposti dalla Legge di Amdahl.

Analisi delle Configurazioni Ibride

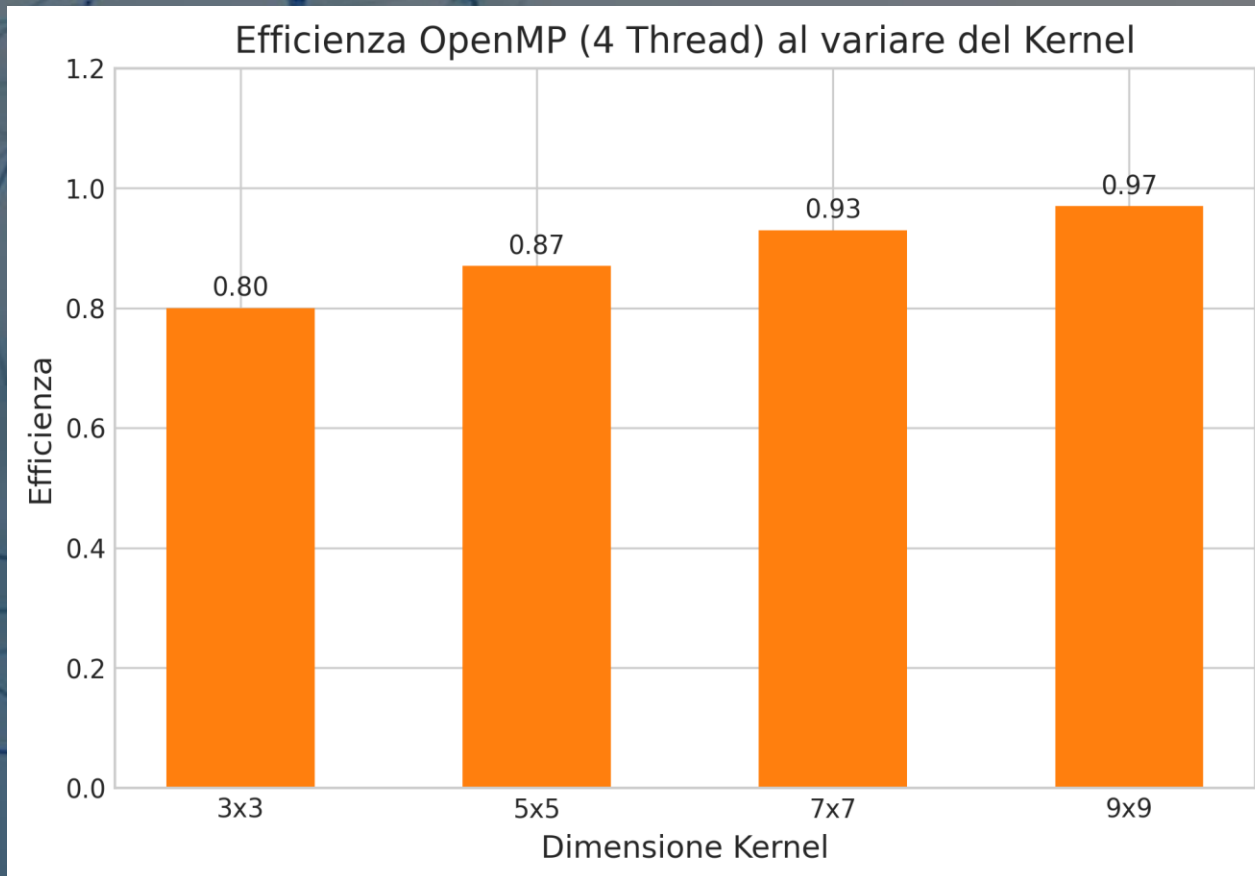


Le configurazioni a dominanza MPI (4x1 e 2x2) offrono prestazioni ottimali (Efficienza > 91%).

La configurazione 1 MPI x 4 OMP subisce un grave crollo prestazionale (**58.8%**).

Il blocco (lock) di sicurezza introdotto da `MPI_Init_thread` crea un severo overhead di sincronizzazione che non viene ammortizzato dallo speedup locale dei thread.

Variazione della Dimensione del Kernel (K)



Aumentando l'area del filtro (da 3x3 a 9x9), cresce la mole di calcoli matematici richiesti per singola lettura in RAM.

L'efficienza parallela di OpenMP (a 4 thread) scala in modo progressivo, passando dall'**80%** (K=3) fino a raggiungere un quasi perfetto **97%** (K=9).

Il problema smette di essere limitato dalla banda di memoria (*memory-bound*) per diventare puramente computazionale (*compute-bound*).